

Universidade NOVA de Lisboa
NOVA School of Science and Technology

Mestrado em Engenharia Informática

3S: transmissão segura de vídeo

Trabalho individual

André Filipe de Sousa Vaz, nº 79264

Unidade curricular

Segurança e Sistemas de Computadores

31 de outubro de 2026

Índice

1 Objetivo e arquitetura **1**

2 Modelo de sistema e ameaças **1**

3 Handshake e estabelecimento de chaves **2**

3.1 Enquadramento e mensagens 2

3.2 Da PSK às chaves de tráfego 2

4 Proteção dos registos e configuração **3**

4.1 Dados, sequência e terminação 3

4.2 Construções criptográficas 3

4.3 Validação do XML 3

5 Integração HTTP, métricas e observações **4**

5.1 Responsabilidades e execução 4

5.2 O que as métricas medem 4

6 Validação funcional e adversarial **5**

6.1 Método e resultados funcionais 5

6.2 Interpretação dos resultados 5

7 Compromisso de chaves e limites **6**

8 Conclusão **6**

Resumo

Este trabalho desenvolve um canal seguro entre um proxy e um servidor de vídeo MP4, mantendo a reprodução num browser. A partir da aplicação de referência, foram acrescentados um handshake autenticado e um protocolo de transporte que verifica cada registo antes de encaminhar o seu conteúdo. O objetivo é proteger a confidencialidade e a integridade do vídeo no troço proxy–servidor, detetando também repetições, alterações de ordem e transmissões incompletas.

O handshake usa segredos previamente partilhados para autenticar os intervenientes e Diffie–Hellman efémero para estabelecer material de sessão. As chaves são derivadas separadamente por fase, direção e função. A configuração XML permite selecionar AES-GCM, ChaCha20-Poly1305 ou AES-CTR com HMAC, sem recompilar, e é validada antes de iniciar a comunicação. A integração preserva pedidos HTTP Range, necessários para procurar posições do vídeo sem transferir todo o ficheiro.

A validação da implementação abrange primitivas criptográficas, XML, handshake, streaming e integração HTTP. Os testes adversariais verificam a rejeição de mensagens manipuladas; uma transferência integral compara o conteúdo recebido com o original e permite observar o custo do formato protegido. São ainda discutidas as consequências do compromisso de chaves e as limitações relativas à disponibilidade e à ligação HTTP ao browser.

Palavras-chave: streaming seguro; DH autenticado; PSK; AEAD; Encrypt-then-MAC.

1 Objetivo e arquitetura

O Work Assignment 1 pede a proteção do troço Secure Proxy–Secure Stream Server, com parâmetros criptográficos externos ao código [1]. A referência Java foi analisada e adaptada para Kotlin/JVM, uma linguagem já conhecida, usando as primitivas JCA do JDK 21.

O browser obtém a página no proxy e pede um vídeo ou intervalo de bytes. O proxy abre uma ligação TCP ao servidor, executa o handshake e verifica os registos recebidos antes de encaminhar o vídeo por HTTP. Cada pedido usa uma sessão nova. O catálogo também é transportado pelo canal protegido.

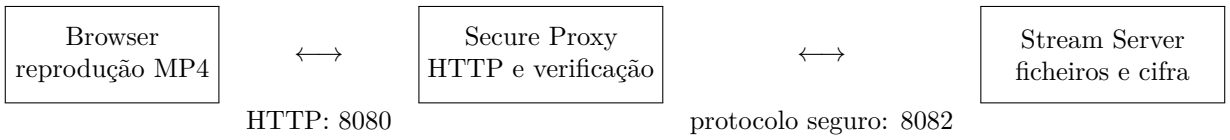


Figura 1: A proteção criptográfica cobre o troço proxy–servidor. A ligação ao browser continua a usar HTTP. As portas são configuráveis.

2 Modelo de sistema e ameaças

O atacante pode observar, intercetar, alterar, inserir, repetir, reordenar ou remover mensagens entre proxy e servidor. Pretende-se proteger a confidencialidade, detetar alterações e rejeitar mensagens fora de ordem ou de outra sessão. Assume-se que endpoints, gerador aleatório e primitivas criptográficas não estão comprometidos [2].

Os segredos iniciais são provisionados fora do protocolo. A autenticação comprova a sua posse; não identifica pessoas. O Authentication Server e o login pertencem à segunda parte do trabalho. Não é garantida a disponibilidade perante interrupções de rede ou consumo de recursos, nem a confidencialidade do troço HTTP entre browser e proxy.

3 Handshake e estabelecimento de chaves

3.1 Enquadramento e mensagens

O cabeçalho tem 9 bytes: versão (2), release (2), tipo (1) e comprimento do payload (4), em ordem big endian. A implementação usa a versão 1:1 e sequências de 8 bytes. O codec acumula leituras parciais de TCP e rejeita comprimentos fora do limite antes de reservar o payload. Um fim de ligação a meio de uma mensagem é um erro.

A Tabela 1 apresenta o formato real. H_i representa o cabeçalho da mensagem i; h_1, h_{12} e h_{123} são SHA-256 dos frames completos, pela ordem de transmissão. N_P e N_S são valores aleatórios públicos de 32 bytes; SRV é o desafio de 32 bytes enviado dentro da mensagem 2. $A = g^a$ e $B = g^b$ são as públicas DH; X é a sequência inicial aleatória.

Tipo	Direção	Payload e proteção
1	Proxy → servidor	$X \parallel PCC \parallel N_P \parallel A \parallel T_1$; HMAC sobre H_1 e todos os campos anteriores à tag.
2	Servidor → proxy	$N_S \parallel \text{Protect}_{K_2}(X+1 \parallel SRV \parallel CCC \parallel B)$; contexto autenticado $H_2 \parallel N_S \parallel h_1$.
3	Proxy → servidor	$\text{Protect}_{K_3}(X+2 \parallel \text{SHA256}(SRV) \parallel MEDIA \parallel Range?)$; contexto $H_3 \parallel h_{12}$.
255	Ambos	Alerta de falha do handshake; termina a tentativa.

Tabela 1: Extensões ao protocolo de referência: DH públicas, randoms, tags, contexto e Range opcional. Protect usa a construção selecionada no XML.

PCC (proposta) e CCC (confirmação) começam pela string `ALG/MODE/IV/Padding`, seguida dos tamanhos, MAC e parâmetros do handshake. Strings UTF-8 e públicas DH têm prefixos de comprimento de 4 bytes. O servidor verifica a tag da mensagem 1 e compara a proposta com a política local antes do DH. A mensagem 2 confirma a mesma configuração; não há negociação de listas nem fallback para um algoritmo mais fraco.

3.2 Da PSK às chaves de tráfego

Cada extremo gera uma chave privada DH nova por ligação, no grupo `ffdhe2048` [4]. A chave pública recebida é validada quanto ao grupo, intervalo e pertença ao subgrupo. O acordo calcula o mesmo segredo $Z = g^{ab}$ nos dois lados. A representação X.509 da chave pública é apenas uma codificação; não envolve certificados.

HKDF-SHA-256 [3] separa as funções das chaves. Nos perfis AEAD, a raiz de autenticação é `SKey`. Nos perfis CTR, os segredos independentes `SKey` e `MacKey` são combinados por HKDF nessa raiz. A derivação segue quatro etapas:

1. A raiz e N_P derivam a chave HMAC-SHA-256 de T_1. Conhecer o random público não permite reproduzir a tag sem o segredo.
2. A raiz, N_P, N_S e h_1 derivam a chave, IV e, se necessário, chave MAC de proteção da mensagem 2. Esta fase ainda não depende de Z: o proxy só recebe B nessa mensagem. N_S torna a resposta nova mesmo perante uma repetição da mensagem 1.
3. A raiz, Z, N_P, SRV e h_{12} produzem um segredo intermédio de sessão. Dele são derivadas a chave e o IV exclusivos da mensagem 3.
4. Após a mensagem 3, rótulos distintos e h_{123} derivam chaves e bases de IV separadas por direção e função: cifra e, no CTR, MAC.

A autenticação vincula as públicas DH e a configuração à troca de mensagens: substituir uma pública exige falsificar uma tag. A resposta ao desafio e a mensagem 3 protegida confirmam ao servidor a participação do proxy nesta sessão. O proxy confirma a aceitação pelo servidor quando recebe os primeiros metadados protegidos; enviar a mensagem 3, por si só, não comprova essa aceitação. Alertas anteriores à sessão não são autenticados e nunca autorizam continuação ou downgrade.

4 Proteção dos registos e configuração

4.1 Dados, sequência e terminação

Depois do handshake, o servidor envia mensagens de tipo 4 até à mensagem final de tipo 5. O primeiro tipo 4 contém metadados; os seguintes contêm blocos DATA de até 1024 bytes. O subtipo faz parte do conteúdo cifrado. Os formatos são:

$$\text{Tipo 4: } q \parallel \text{Protect}(\text{subtipo} \parallel \text{conteúdo}),$$
$$\text{Tipo 5: } q \parallel \text{Protect}(q \parallel \texttt{END} \parallel \text{comprimento} \parallel \text{random}).$$

A primeira sequência é $X+3$, incluindo os metadados; cada registo consome a seguinte. O random de END é uma string hexadecimal de 64 caracteres obtida de 32 bytes aleatórios. Cabeçalho, sequência exterior e identificador de sessão h_{123} são autenticados. Assim, não basta copiar um bloco válido de outra posição ou sessão.

O recetor exige a sequência exata, autentica o registo e valida o subtipo antes de usar o conteúdo. Confere ainda o comprimento anunciado e o END, incluindo a sequência interior. Mantém o último bloco pendente até validar a terminação. A ausência de END impede aceitar uma transmissão completa, sem exigir guardar o ficheiro inteiro em memória. Blocos anteriores já encaminhados não podem ser retirados do browser.

4.2 Construções criptográficas

Construção	Chave de cifra	IV/nonce	Autenticação
AES-GCM	128/192/256 bits	96 bits	Tag integrada de 128 bits
ChaCha20-Poly1305	256 bits	96 bits	Tag integrada de 128 bits
AES-CTR/HMAC	128/192/256 bits	128 bits	HMAC-SHA-256; adicionalmente AES-256 com SHA-384 ou SHA-512

Tabela 2: Nove pares de configurações. HMAC usa a tag completa e chaves independentes com pelo menos 256, 384 ou 512 bits, respetivamente.

AES-GCM e ChaCha20-Poly1305 fornecem cifra autenticada [5, 6]. Em AES-CTR, a integridade é acrescentada por Encrypt-then-MAC: calcula-se $C = E_{K_e}(M)$ e depois $T = \text{HMAC}_{K_m}(D \parallel |AAD| \parallel AAD \parallel IV \parallel C)$, com domínio fixo D. O recetor compara a tag completa antes de decifrar. As chaves de cifra e MAC são derivadas separadamente; não são a mesma chave.

Nos perfis AEAD, o nonce de cada registo resulta da base de IV XOR com a sequência preenchida a 96 bits. No CTR, o contador inicial é $(\text{prefixo}_{64} \oplus q) \parallel 0^{64}$. Cada registo ocupa um domínio de contador diferente; o limite de tamanho impede esgotar os 64 bits inferiores [7]. Nonces diferentes, sem esta separação, não bastariam para evitar sobreposição dos contadores CTR. As chaves das mensagens 2 e 3 são distintas e de uso único. Não é permitido reiniciar um emissor na mesma sessão.

4.3 Validação do XML

Cada pasta de perfil contém um ficheiro por papel, selecionado pelo caminho de configuração. O parser limita o XML a 64 KiB, desativa DTD e entidades externas e rejeita tags, atributos e duplicados inesperados. O validador verifica combinações do catálogo, tamanhos de chaves, parâmetros do handshake e disponibilidade no provider antes de abrir sockets.

Estas construções usam `NoPadding`. O campo `IV=NULL` indica gestão dinâmica; um IV fixo no XML é rejeitado. AEAD rejeita parâmetros de MAC adicional; CTR exige um segredo MAC independente. Os templates não contêm segredos: são preenchidos com bytes aleatórios de `openssl rand`, iguais no par correspondente, e montados externamente aos containers. Mudar entre perfis suportados dispensa recompilação; acrescentar uma construção nova exige implementação e testes.

5 Integração HTTP, métricas e observações

5.1 Responsabilidades e execução

`ProxyHandshake` e `ServerHandshake` coordenam a troca inicial; os codecs tratam exclusivamente da representação binária. `MessageProtection` concentra AEAD e Encrypt-then-MAC. O envio e a receção de registos ficam em `SecureStreamSender` e `SecureStreamReceiver`, incluindo sequência, metadados, DATA e END. Esta separação mantém o fluxo do protocolo independente do perfil escolhido e permite testar cada responsabilidade.

`SecureStreamServer` abre o ficheiro pedido dentro da diretoria de media, com validação do caminho e de symlinks. Os metadados autenticados indicam estado HTTP, tipo de conteúdo, comprimento total e da resposta, intervalo e suporte a Range. `SecureProxyServer` traduz-os para os headers HTTP. São suportadas respostas 200, 206, 404 e 416, incluindo ranges fechados, abertos e por sufixo. Range designa posições de bytes do ficheiro, não números de pacotes. Cada novo pedido inicia um novo handshake.

Falhas antes dos headers HTTP produzem 502; depois dos headers, a ligação é fechada como resposta incompleta. O proxy não envia um segundo estado HTTP. Procurar outra posição no vídeo pode cancelar a resposta anterior e originar `Broken pipe`; esse evento é registado sem ser confundido com uma falha criptográfica.

Proxy e servidor usam pools de threads por ligação. A reutilização de threads simplifica a integração com sockets bloqueantes. Não foi adotada uma reescrita com corrotinas; a pool não impõe um limite global de concorrência. Docker Compose arranca os dois serviços, usando a mesma imagem JRE 21 e mounts de XML/media só de leitura. As verificações de saúde são passivas e não geram handshakes artificiais. A configuração externa também permite executar os processos em hosts diferentes.

5.2 O que as métricas medem

O proxy conta registos DATA autenticados e blocos cuja escrita e flush para o cliente terminaram. Mede os bytes de plaintext e de frames completos do protocolo, incluindo metadados e END, e compara-os com o tamanho original. O volume de protocolo exclui o handshake e os headers TCP/IP.

O tempo é medido com `System.nanoTime`, desde o início da receção do stream, após o handshake, até à última escrita ou falha. Inclui espera de rede, verificação, decifra e escrita HTTP. O throughput é B/t, com unidades decimais: 1 MB = 10^6 bytes. O tempo total de curl inclui conexão e handshake, pelo que mede outro intervalo.

```
[STREAM OK] "sintel.mp4 HTTP 200
enviados 4.37 MB/4.37 MB (original 4.37 MB)
blocos autenticados/encaminhados:  4270/4270
protocolo:  4.52 MB | 361.23 ms | 12.10 MB/s
```

Figura 2: Transcrição do output real de uma transferência completa em 31/10/2026, AES-256-GCM. Apenas a disposição das linhas foi ajustada.

Na observação da Figura 2, em Docker local num host macOS arm64, foram recebidos 4 372 373 bytes. Os SHA-256 do original e do ficheiro obtido com curl coincidiram. Para 4270 blocos, o formato AEAD acrescenta $34N + 182 = 145\,362$ bytes, cerca de 3,32%, totalizando 4 517 735 bytes de protocolo, coerentes com os 4,52 MB apresentados. É uma observação isolada, sem média ou comparação de desempenho entre algoritmos. O pedido `Range: bytes=0-1023` devolveu 206, comprimento 1024 e `Content-Range: bytes 0-1023/4372373`.

6 Validação funcional e adversarial

6.1 Método e resultados funcionais

A suite Kotlin/JUnit foi executada com `./gradlew test`, usando JDK 21, em 31/10/2026: **141 testes, sem falhas, erros ou testes ignorados**. Inclui vetores conhecidos para AES-GCM, ChaCha20-Poly1305, CTR, HMAC e HKDF; parsing e validação XML; codecs; derivação e separação de chaves; transições do handshake; e integração HTTP nos nove pares de perfis. Os testes usam segredos públicos de teste, separados da configuração de execução.

Os testes de integração usam sockets locais e conteúdos temporários para verificar bytes, estados HTTP, ranges e encerramento correto. A reprodução de MP4 reais foi validada no browser. Foi também confirmado o funcionamento simultâneo com três browsers ligados ao mesmo proxy. Essa observação verifica concorrência funcional, mas não equivale a três instâncias independentes do proxy nem a um ensaio entre computadores físicos. Um segundo processo proxy foi verificado contra o mesmo servidor; não se apresenta esse ensaio como medição simultânea de carga.

A classe `adversarialTest` reúne **22 métodos**, com um cenário legítimo de controlo e casos negativos separados. A maioria é exercitada em cinco perfis representativos: AES-256-GCM, ChaCha20-Poly1305 e AES-256-CTR com cada um dos três HMAC. Assim, um método pode executar mais do que uma configuração. A Tabela 3 liga cada ameaça à verificação observada.

Ameaça / casos	Manipulação no teste	Critério de rejeição observado
Impersonação (A1)	PSK errada; em CTR, SKey certa mas MacKey errada.	ClientHello não autentica; handshake abortado com alerta.
MITM no handshake (A2)	Alterar proposta, pública DH válida, tag de 2/3, random público ou ordem inicial.	Tag/contexto ou estado inválido; servidor não inicia o stream.
Alteração do stream (A3)	Modificar ciphertext, tag DATA ou metadados.	Registo não autenticado não é encaminhado; metadados inválidos originam 502.
Replay (A4)	Repetir DATA; reutilizar mensagem 2 ou 3 de outra sessão.	Sequência ou contexto/chaves de sessão não correspondem.
Lacunas e ordem (A5)	Remover um DATA ou trocar dois registos.	Sequência recebida difere da próxima esperada.
Truncamento (A6)	Omitir ou alterar END.	Stream não termina com sucesso; último bloco retido.
Framing abusivo (A7)	Comprimento excessivo, header incompleto ou tipo desconhecido.	Leitura rejeitada; comprimento verificado antes de alocar o payload.

Tabela 3: Resumo dos testes adversariais implementados em Kotlin, sem ferramenta externa de ataque.

6.2 Interpretação dos resultados

Os casos negativos verificam rejeições concretas. Não são uma prova formal de segurança. A resistência à observação passiva depende da cifra e do segredo das chaves; a resistência à substituição DH depende da autenticação do handshake. Uma repetição da mensagem 1 pode provocar uma nova resposta do servidor, mas as mensagens antigas 2 e 3 não concluem a nova sessão.

Num ataque ao stream, bytes autenticados anteriores podem já ter sido entregues. A propriedade testada é não libertar o registo inválido e não aceitar uma terminação falsa. Num cancelamento pelo browser, os contadores podem diferir: ao autenticar o bloco 4, o recetor pode falhar ao escrever o bloco 3 pendente, tendo completado apenas duas escritas. Um resultado 4/2 representa esse estado e não a duplicação de dois pacotes TCP. Numa transferência concluída com sucesso, os contadores coincidem.

7 Compromisso de chaves e limites

Material de tráfego de uma sessão exposto. A confidencialidade e a possibilidade de autenticar mensagens da direção correspondente ficam comprometidas se o atacante conhecer todo esse material. Em CTR, conhecer só a chave de cifra não permite calcular o HMAC; conhecer só a chave MAC não revela o plaintext. A separação por rótulo e direção, o transcript e um DH novo em cada ligação evitam reutilizar o mesmo material entre sessões.

Segredos de autenticação expostos posteriormente. Conhecer SKey nos perfis AEAD, ou SKey e MacKey nos perfis CTR, permite reconstruir a proteção inicial de uma gravação do handshake e obter as públicas DH. Não fornece, por si só, Z. A proteção de sessões passadas depende da dificuldade do problema DH e de as privadas efémeras já não estarem disponíveis. Este é o argumento de forward secrecy do desenho, não uma conclusão demonstrada por testes de round-trip. A limpeza de arrays é feita quando possível, mas a JVM não garante a eliminação de todas as cópias de material sensível.

Sessões futuras e endpoints. A exposição dos segredos de autenticação permite novas tentativas de impersonação; não há recuperação automática após compromisso. É necessário substituir os segredos por um canal externo. Um endpoint comprometido pode aceder ao vídeo e às chaves que processa. Não é prometida proteção contra esse cenário, contra exaustão da pool de threads ou contra interrupções deliberadas da rede.

8 Conclusão

A solução mantém a reprodução MP4 e o suporte a HTTP Range, acrescentando autenticação por segredos partilhados, DH efémero e proteção por registo. Nove configurações externas reutilizam o mesmo framing e a mesma máquina de estados. Os testes confirmam a integração e a rejeição das manipulações exercitadas; uma transferência integral confirma a igualdade de conteúdo e ilustra as métricas recolhidas.

As principais escolhas foram separar configuração, codecs, handshake e stream; usar AEAD quando disponível e Encrypt-then-MAC para CTR; e validar cada registo antes de o encaminhar. O compromisso é manter um protocolo e execução simples, com um handshake por pedido HTTP e threads bloqueantes. Como trabalho adicional ficam o controlo global de recursos, medições repetidas entre hosts e a proteção do troço HTTP. Provisioning de segredos e autenticação de utilizadores continuam fora deste protocolo.

Referências

[1] Segurança e Sistemas de Computadores, NOVA FCT. *WA1: 3S – Secure Streaming Solution, Part I* e *Additional Information / FAQ*. 2026. Enunciado e instruções de avaliação.

[2] Materiais da unidade curricular. *Applied Cryptography* e *Complementary Applied Cryptography*. Cifra simétrica, MAC, AEAD e acordo de chaves.

[3] H. Krawczyk e P. Eronen. *HMAC-based Extract-and-Expand Key Derivation Function*. RFC 5869, 2010. https://www.rfc-editor.org/rfc/rfc5869

[4] D. Gillmor. *Negotiated Finite Field Diffie-Hellman Ephemeral Parameters*. RFC 7919, 2016, apêndice A.1: ffdhe2048. https://www.rfc-editor.org/rfc/rfc7919

[5] NIST. *Recommendation for Block Cipher Modes of Operation: GCM and GMAC*. SP 800-38D, 2007. https://doi.org/10.6028/NIST.SP.800-38D

[6] Y. Nir e A. Langley. *ChaCha20 and Poly1305 for IETF Protocols*. RFC 8439, 2018. https://www.rfc-editor.org/rfc/rfc8439

[7] NIST. *Recommendation for Block Cipher Modes of Operation: Methods and Techniques*. SP 800-38A, 2001, apêndices B e F.5. https://doi.org/10.6028/NIST.SP.800-38A